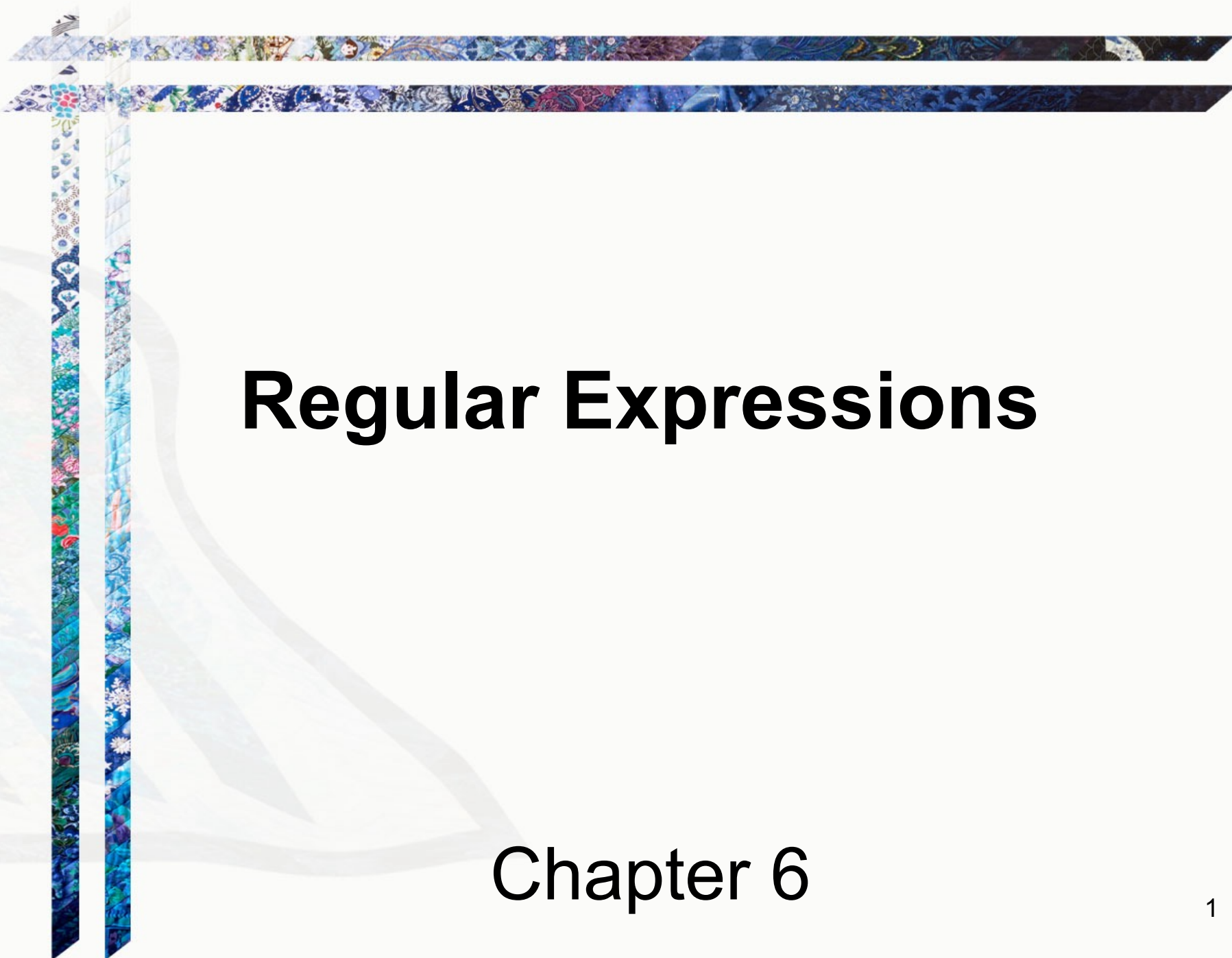
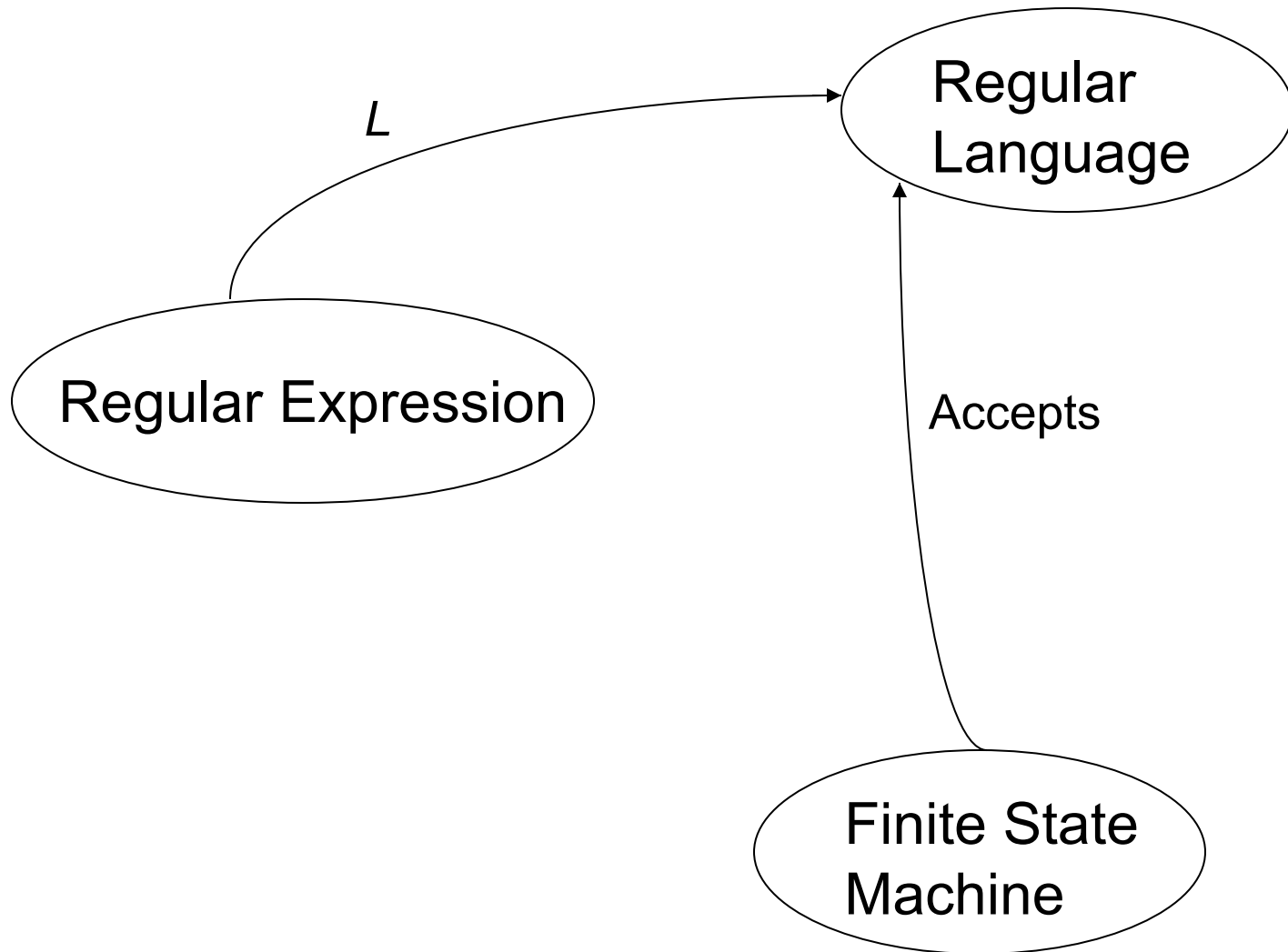


A decorative border on the left side of the slide, consisting of two vertical strips and two horizontal strips. The strips feature intricate, colorful patterns in shades of blue, green, red, and gold, resembling traditional textile or tapestry designs. The background of the slide is white with a faint, light gray abstract shape in the lower-left corner.

# Regular Expressions

## Chapter 6

# Regular Languages



# Regular Expressions

The **regular expressions** over an alphabet  $\Sigma$  are all and only the strings that can be obtained as follows:

1.  $\emptyset$  is a regular expression.
2.  $\varepsilon$  is a regular expression.
3. Every element  $a \in \Sigma$  is a regular expression.
4. If  $\alpha, \beta$  are regular expressions, then so is  $\alpha\beta$ .
5. If  $\alpha, \beta$  are regular expressions, then so is  $\alpha \cup \beta$ .
6. If  $\alpha$  is a regular expression, then so is  $\alpha^*$ .
7.  $\alpha$  is a regular expression, then so is  $\alpha^+$ .
8. If  $\alpha$  is a regular expression, then so is  $(\alpha)$ .

Sometimes union ' $\cup$ ' is also denoted as '+' or '|'.



# Regular Expression Examples

If  $\Sigma = \{a, b\}$ , the following are regular expressions:

$\emptyset$

$\varepsilon$

$a$

$(a \cup b)^*$

$abba \cup \varepsilon$

# Regular Expressions Define Languages

Semantic interpretation: the **language  $L(\alpha)$**  expressed by a regular expression  $\alpha$ :

1.  $L(\emptyset) = \emptyset$ .
2.  $L(\varepsilon) = \{\varepsilon\}$ .
3.  $L(c) = \{c\}$ , where  $c \in \Sigma$ .
4.  $L(\alpha\beta) = L(\alpha) L(\beta)$ .
5.  $L(\alpha \cup \beta) = L(\alpha) \cup L(\beta)$ .
6.  $L(\alpha^*) = (L(\alpha))^*$ .
7.  $L(\alpha^+) = L(\alpha\alpha^*) = L(\alpha) (L(\alpha))^*$ . If  $L(\alpha)$  is equal to  $\emptyset$ , then  $L(\alpha^+)$  is also equal to  $\emptyset$ . Otherwise  $L(\alpha^+)$  is the language that is formed by concatenating together one or more strings drawn from  $L(\alpha)$ .
8.  $L((\alpha)) = L(\alpha)$ .

# Example

$$\begin{aligned} L((a \cup b)^*b) &= L((a \cup b)^*) L(b) \\ &= (L((a \cup b)))^* L(b) \\ &= (L(a) \cup L(b))^* L(b) \\ &= (\{a\} \cup \{b\})^* \{b\} \\ &= \{a, b\}^* \{b\}. \end{aligned}$$

# Examples

$$L( a^*b^* ) =$$

$$L( (a \cup b)^* ) =$$

$$L( (a \cup b)^*a^*b^* ) =$$

$$L( (a \cup b)^*abba(a \cup b)^* ) =$$

# Going the Other Way

$$L = \{w \in \{a, b\}^*: |w| \text{ is even}\}$$

# Going the Other Way

$$L = \{w \in \{a, b\}^*: |w| \text{ is even}\}$$

$$((a \cup b) (a \cup b))^*$$

$$(aa \cup ab \cup ba \cup bb)^*$$

# Going the Other Way

$$L = \{w \in \{a, b\}^*: |w| \text{ is even}\}$$

$$((a \cup b) (a \cup b))^*$$

$$(aa \cup ab \cup ba \cup bb)^*$$

$$L = \{w \in \{a, b\}^*: w \text{ contains an odd number of } a' \text{ s}\}$$

# Going the Other Way

$$L = \{w \in \{a, b\}^*: |w| \text{ is even}\}$$

$$((a \cup b) (a \cup b))^*$$

$$(aa \cup ab \cup ba \cup bb)^*$$

$$L = \{w \in \{a, b\}^*: w \text{ contains an odd number of } a' \text{ s}\}$$

$$b^* (ab^*ab^*)^* a b^*$$

$$b^* a b^* (ab^*ab^*)^*$$



# More Regular Expression Examples

$$L ( (aa^*) \cup \varepsilon ) =$$

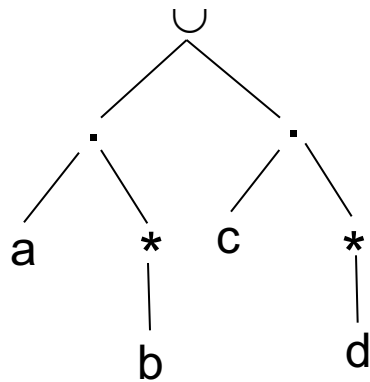
$$L ( (a \cup \varepsilon)^* ) =$$

$$L = \{w \in \{a, b\}^* : \text{there is no more than one } b \text{ in } w\}$$

$$L = \{w \in \{a, b\}^* : \text{no two consecutive letters in } w \text{ are the same}\}$$

# Operator Precedence in Regular Expressions

|         | Regular Expressions | Arithmetic Expressions |
|---------|---------------------|------------------------|
| Highest | Kleene star         | exponentiation         |
|         | concatenation       | multiplication         |
| Lowest  | union               | addition               |



$ab^* \cup cd^*$

$x y^2 + i j^2$

# The Details Matter

$$a^* \cup b^* \neq (a \cup b)^*$$

$$(ab)^* \neq a^*b^*$$



# The Details Matter

$L_1 = \{w \in \{a, b\}^* : \text{every } a \text{ is immediately followed a } b\}$

A regular expression for  $L_1$ :

A FSM for  $L_1$ :

$L_2 = \{w \in \{a, b\}^* : \text{every } a \text{ has a matching } b \text{ somewhere}\}$

A regular expression for  $L_2$ :

A FSM for  $L_2$ :



# Kleene's Theorem

Finite state machines and regular expressions define the same class of languages.

***Theorem 6.3 (Kleene):*** The class of languages that can be defined with regular expressions is exactly the class of regular languages.



# For Every Regular Expression There is a Corresponding FSM

**Proof:** By construction (**Thompson's construction:** simpler than the one in textbook).

We shall build a NDFSM for each regular expression such that:

- its starting state has no incoming edges
- it has only one accepting state that has no outgoing edges

We use structural induction:

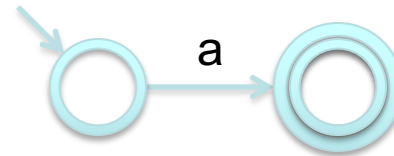
# For Every Regular Expression There is a Corresponding FSM

NDFSM's for the basic blocks:

$\emptyset$ :

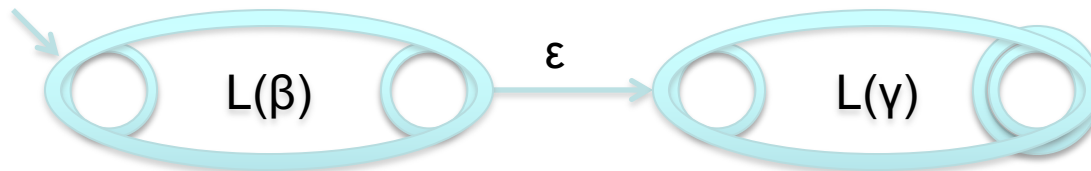


A single element  $a \in \Sigma$ :



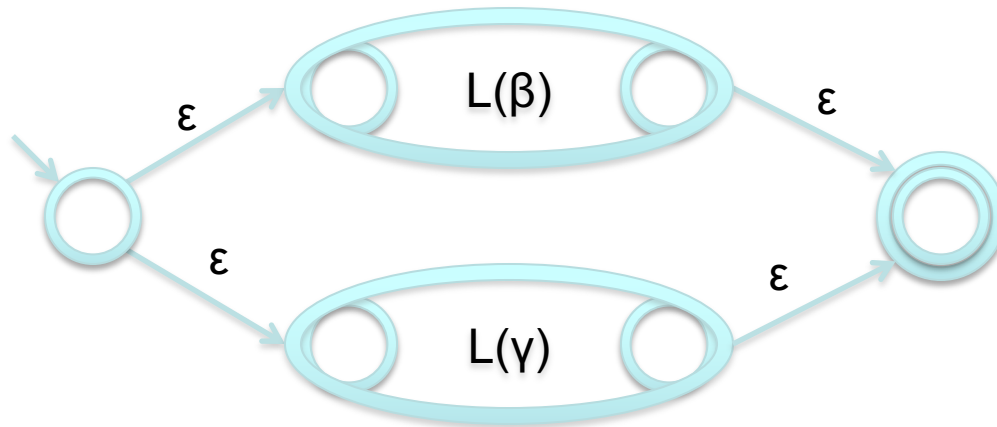
# Concatenation

$\alpha = \beta \gamma$ : build a NDFSM for  $L(\alpha)$



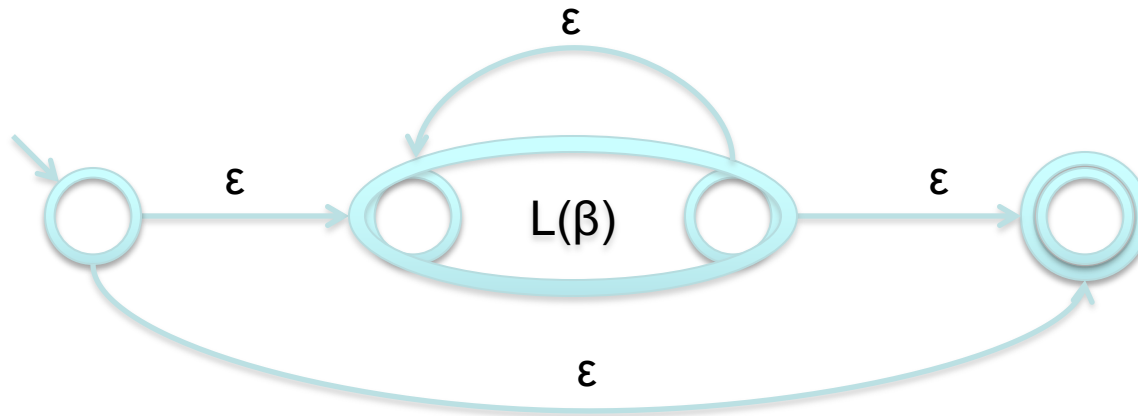
# Union

$\alpha = \beta \cup \gamma$ : build a NDFSM for  $L(\alpha)$



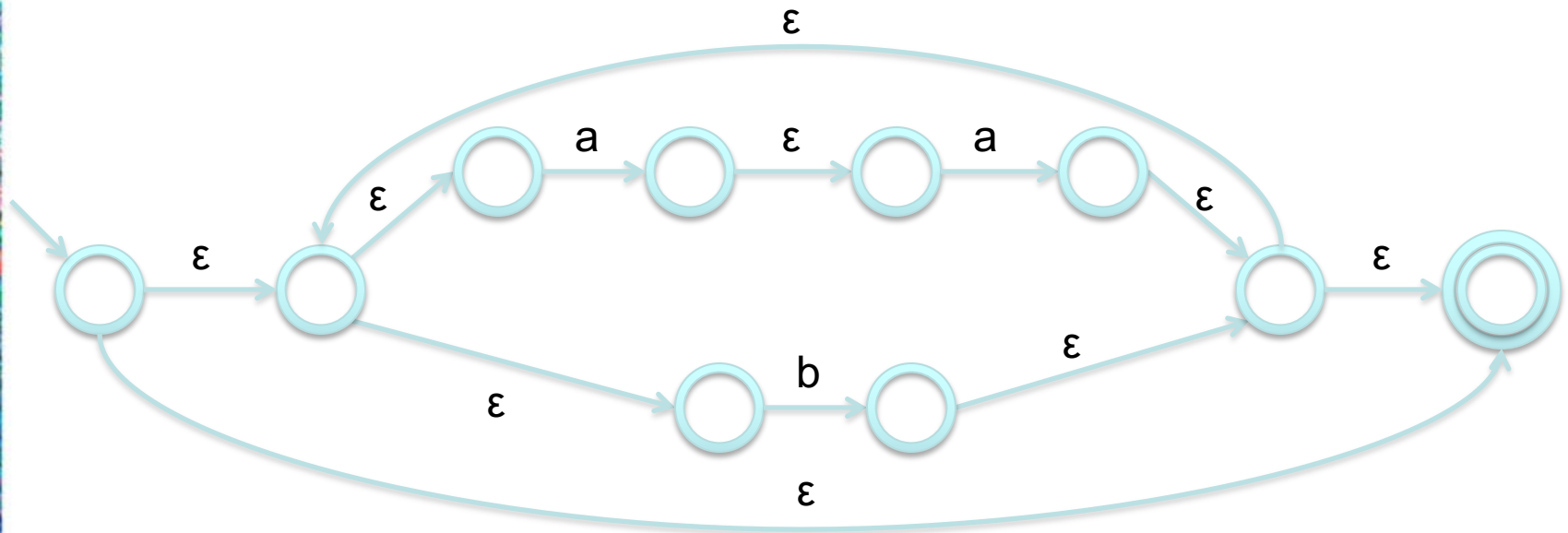
# Kleene \*

$\alpha = \beta^*$ : build a NDFSM for  $L(\alpha)$



# Example

$$\alpha = (aa \cup b)^*$$





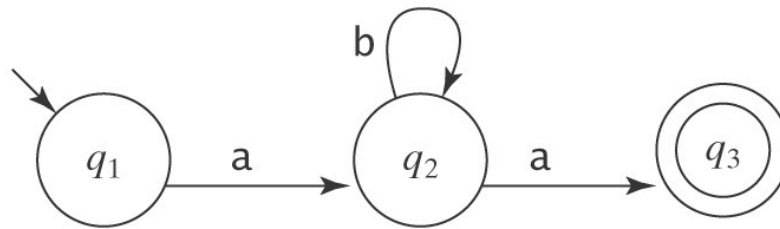
# For Every FSM There is a Corresponding Regular Expression

We' ll show this by **construction**.

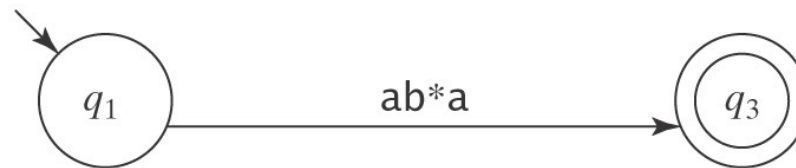
The key idea is that we' ll allow arbitrary regular expressions to label the transitions of an FSM.

# A Simple Example

Let  $M$  be:



Suppose we rip out state 2:

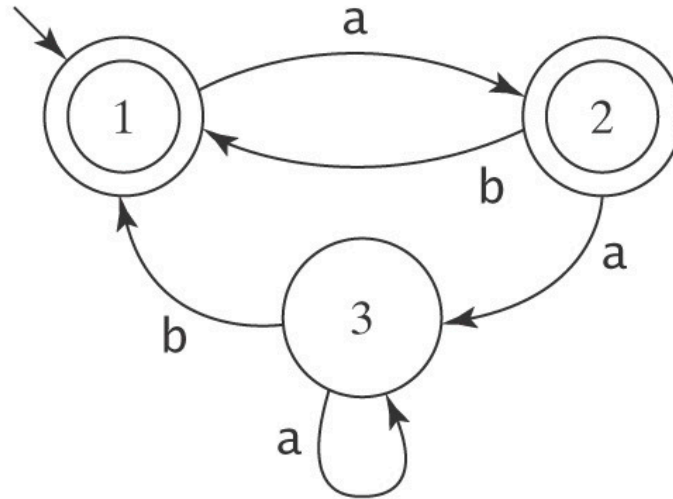


# The Algorithm *fsmtoregexheuristic*

*fsmtoregexheuristic*( $M$ : FSM) =

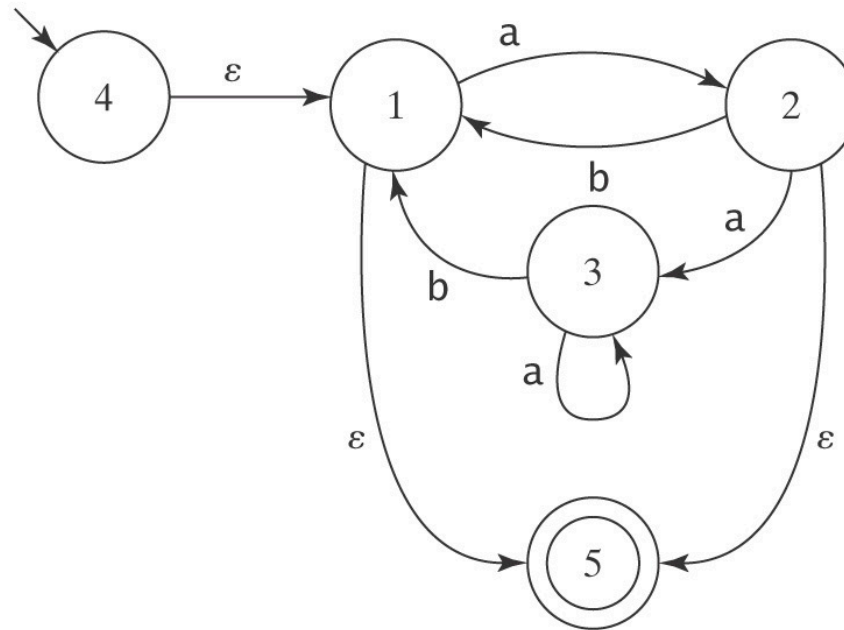
1. Remove unreachable states from  $M$ .
2. If  $M$  has no accepting states then return  $\emptyset$ .
3. If the start state of  $M$  is part of a loop, create a new start state  $s$  and connect  $s$  to  $M$ 's start state via an  $\varepsilon$ -transition.
4. If there is more than one accepting state of  $M$  or there are any transitions out of any of them, create a new accepting state and connect each of  $M$ 's accepting states to it via an  $\varepsilon$ -transition. The old accepting states no longer accept.
5. If  $M$  has only one state then return  $\varepsilon$ .
6. Until only the start state and the accepting state remain do:
  - 6.1 Select *rip* (not  $s$  or an accepting state).
  - 6.2 Remove *rip* from  $M$ .
  - 6.3 \*Modify the transitions among the remaining states so  $M$  accepts the same strings.
7. Return the regular expression that labels the one remaining transition from the start state to the accepting state.

# An Example



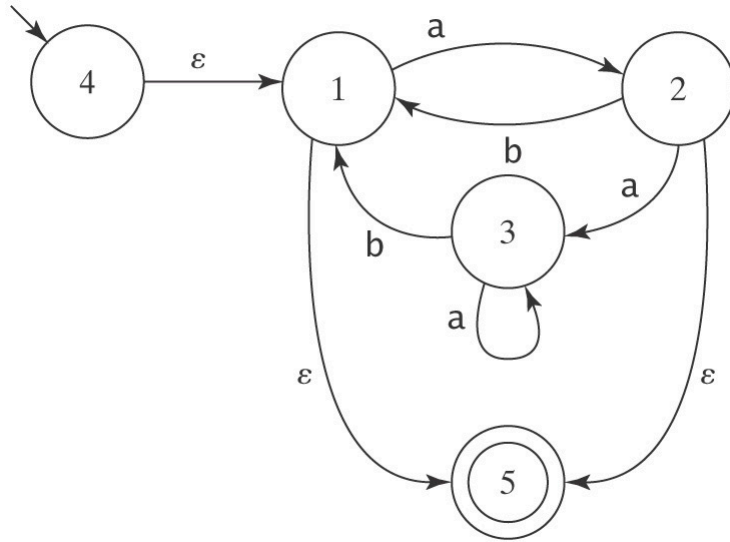
1. Create a new initial state and a new, unique accepting state, neither of which is part of a loop.

# An Example, Continued

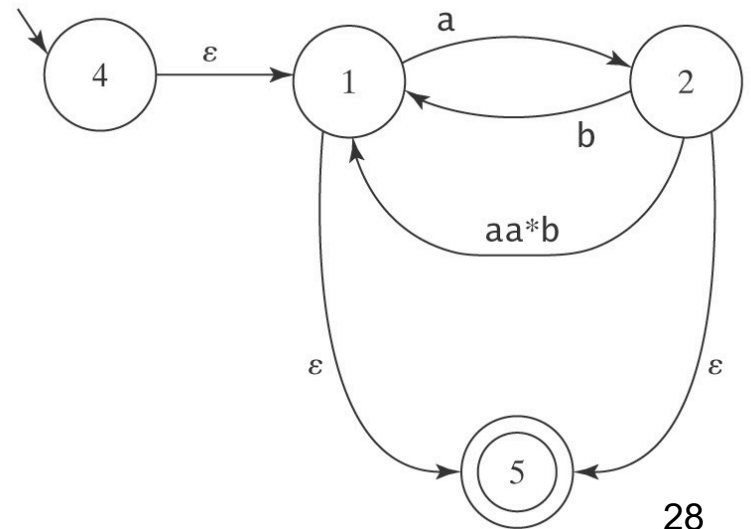


2. Remove states and arcs and replace with arcs labelled with larger and larger regular expressions.

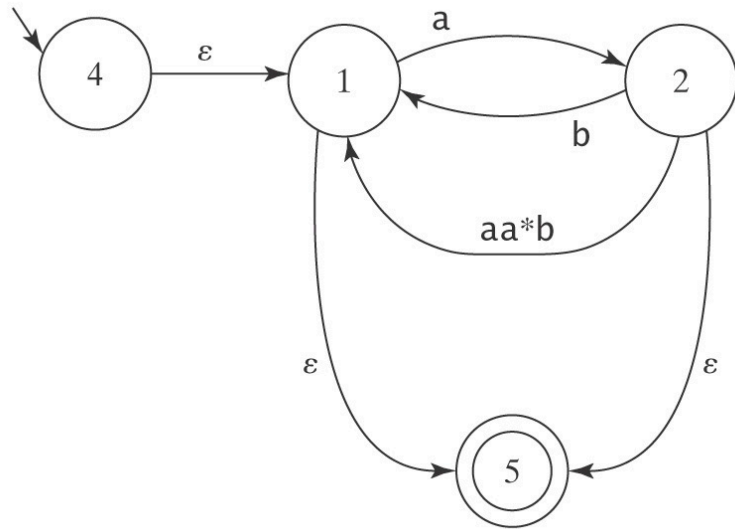
# An Example, Continued



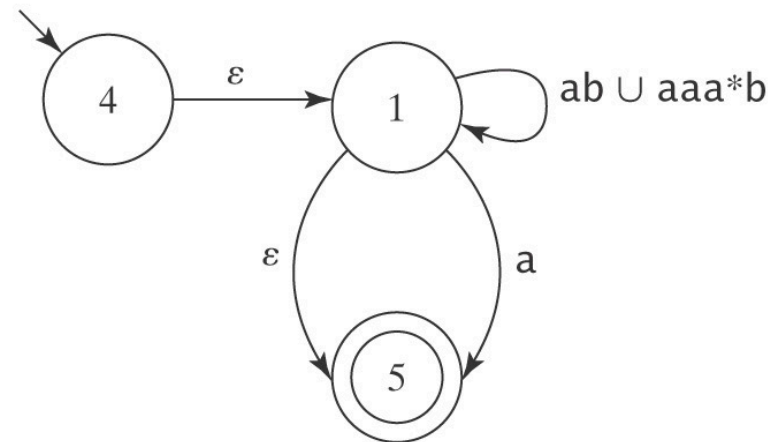
Remove state 3:



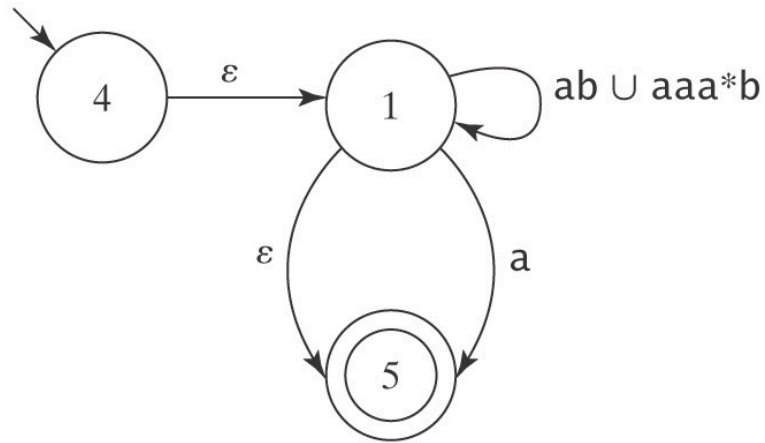
# An Example, Continued



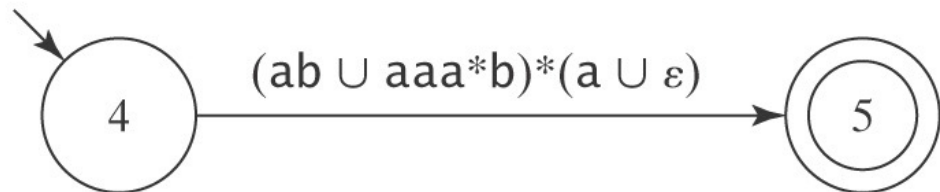
Remove state 2:



# An Example, Continued



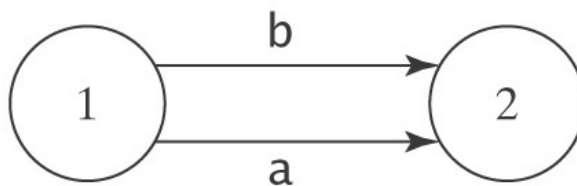
Remove state 1:



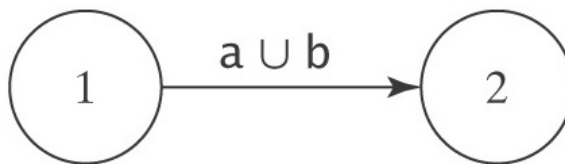
# Further Modifications to $M$ Before We Start

We require that, from every state other than the accepting state there must be exactly one transition to every state (including itself) except the start state. And into every state other than the start state there must be exactly one transition from every state (including itself) except the accepting state.

1. If there is more than one transition between states  $p$  and  $q$ , **collapse** them into a single transition:

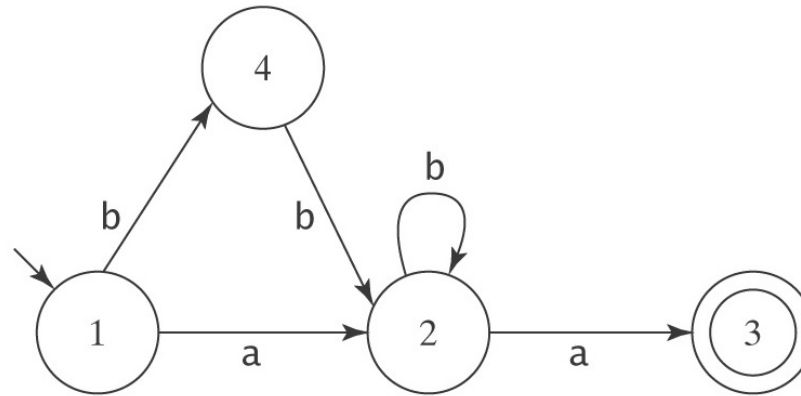


becomes:

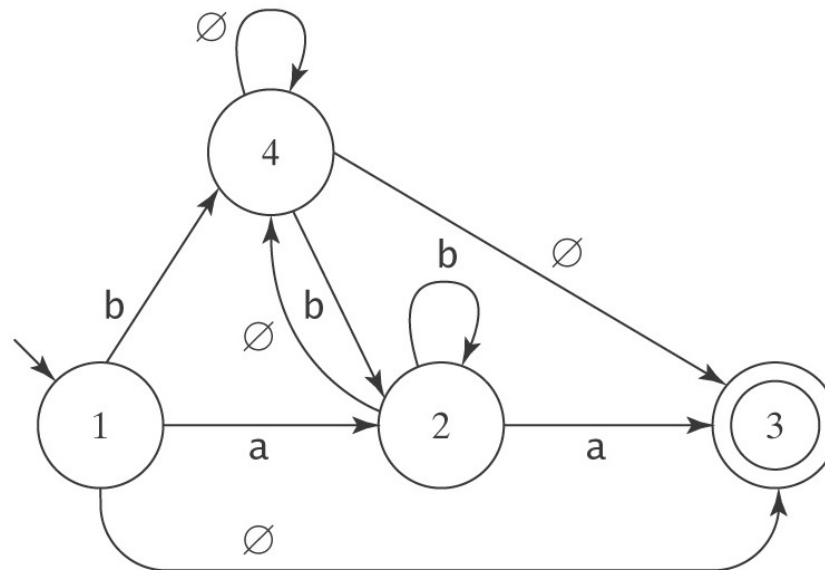


# Further Modifications to $M$ Before We Start

2. If any of the required transitions are missing, **add** them:

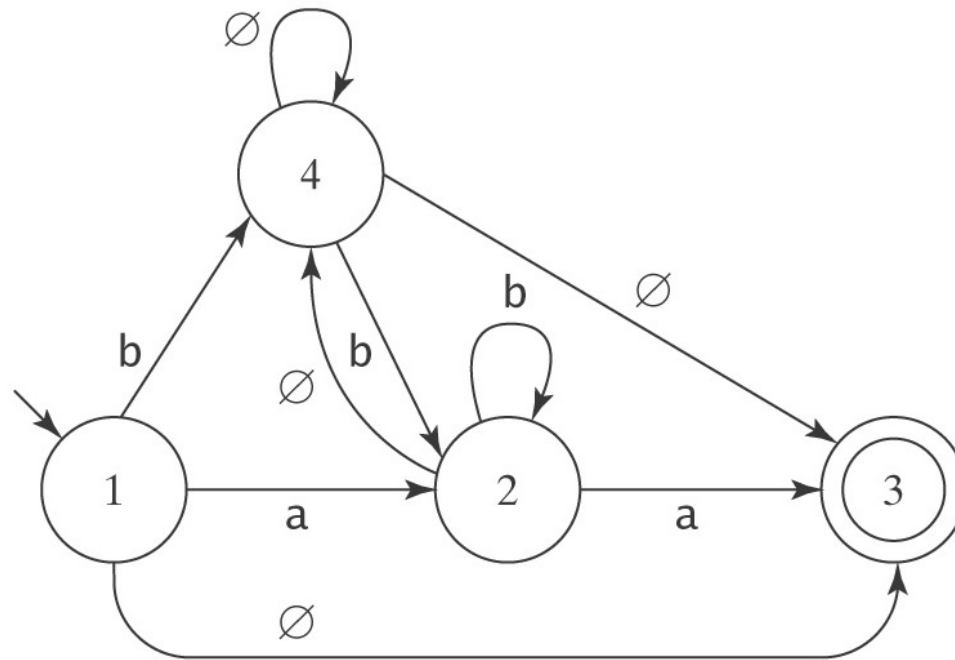


becomes:



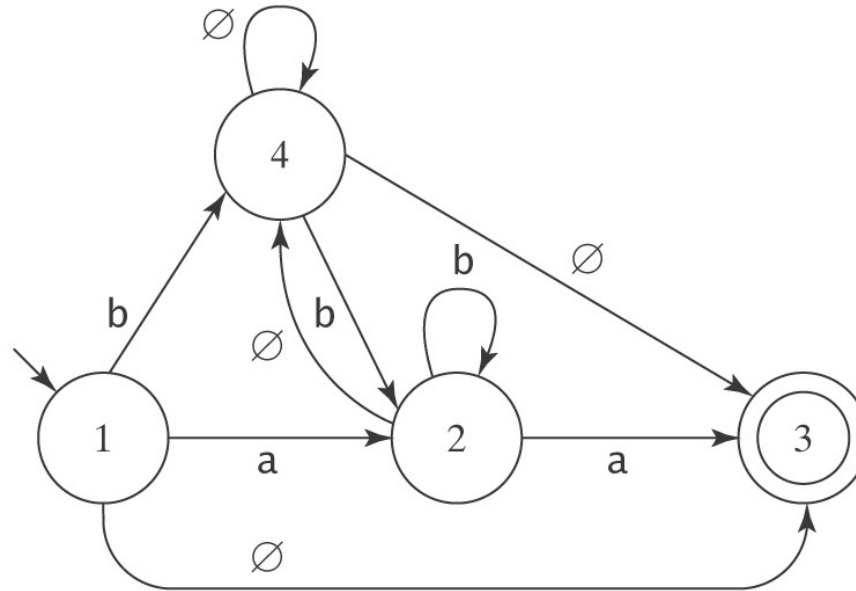
# Ripping Out States

3. Choose a state. **Rip** it out. **Restore** functionality.



Suppose we rip state 2.

# What Happens When We Rip?



Consider any pair of states  $p$  and  $q$ . Once we remove *rip*, how can  $M$  get from  $p$  to  $q$ ?

- It can still take the transition that went directly from  $p$  to  $q$ , or
- It can take the transition from  $p$  to *rip*. Then, it can take the transition from *rip* back to itself zero or more times. Then it can take the transition from *rip* to  $q$ .

# Defining $R(p, q)$

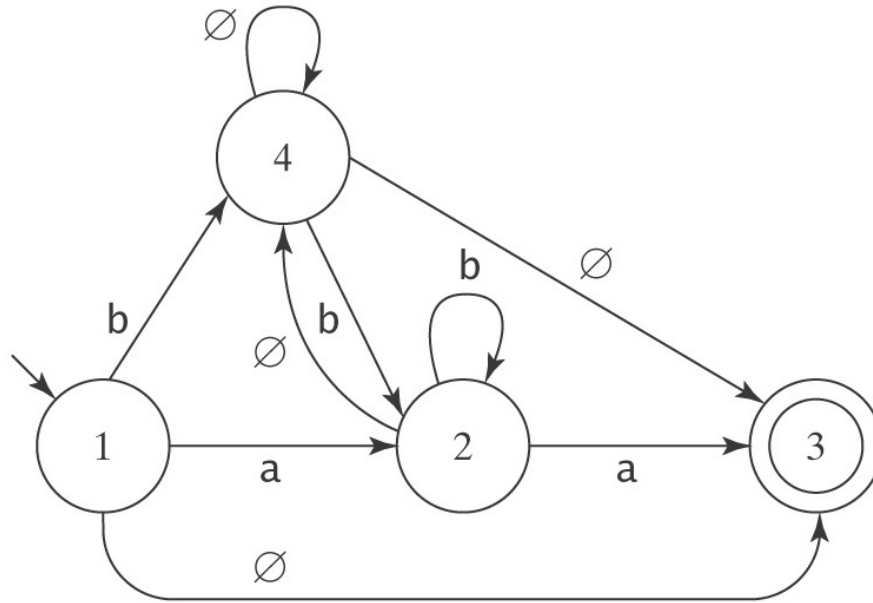
After removing  $rip$ , the new regular expression that should label the transition from  $p$  to  $q$  is:

$$\begin{array}{ll} R(p, q) & /* \text{ Go directly from } p \text{ to } q \\ \cup & /* \text{ or} \\ R(p, rip) & /* \text{ Go from } p \text{ to } rip, \text{ then} \\ R(rip, rip)^* & /* \text{ Go from } rip \text{ back to itself} \\ & \text{any number of times, then} \\ R(rip, q) & /* \text{ Go from } rip \text{ to } q \end{array}$$

Without the comments, we have:

$$R'(p, q) = R(p, q) \cup R(p, rip) R(rip, rip)^* R(rip, q)$$

# Returning to Our Example



$$R' = R(p, q) \cup R(p, rip) R(rip, rip)^* R(rip, q)$$

Let  $rip = 2$ . Then:

$$\begin{aligned} R'(1, 3) &= R(1, 3) \cup R(1, rip) R(rip, rip)^* R(rip, 3) \\ &= R(1, 3) \cup R(1, 2) R(2, 2)^* R(2, 3) \\ &= \emptyset \cup a \quad b^* \quad a \\ &= ab^*a \end{aligned}$$

# The Algorithm *fsmtoregex*

*fsmtoregex*( $M$ : FSM) =

1.  $M' = \text{standardize}(M$ : FSM).
2. Return *buildregex*( $M'$ ).

*standardize*( $M$ : FSM) =

1. **Remove unreachable** states from  $M$ .
2. If necessary, create a **new start** state.
3. If necessary, create a **new accepting** state.
4. If there is more than one transition between states  $p$  and  $q$ , **collapse** them.
5. If any transitions are missing, **create** them with label  $\emptyset$ .

# The Algorithm *fsmtoregex*

*buildregex*(*M*: FSM) =

1. If *M* has no accepting states then return  $\emptyset$ .
2. If *M* has only one state, then return  $\varepsilon$ .
3. **Until only the start and accepting states remain** do:
  - 3.1 Select some state *rip* of *M*.
  - 3.2 For every transition from *p* to *q*, if both *p* and *q* are not *rip* then do  
Compute the new label *R'* for the transition  
from *p* to *q*:

$$R'(p, q) = R(p, q) \cup R(p, rip) R(rip, rip)^* R(rip, q)$$

- 3.3 **Remove** *rip* and all transitions into and out of it.
4. Return the regular expression that labels the transition from the start state to the accepting state.<sup>38</sup>

# Regular Expressions in Perl

| <i>Syntax</i>    | <i>Name</i>     | <i>Description</i>                                                                                         |
|------------------|-----------------|------------------------------------------------------------------------------------------------------------|
| <i>abc</i>       | Concatenation   | Matches <i>a</i> , then <i>b</i> , then <i>c</i> , where <i>a</i> , <i>b</i> , and <i>c</i> are any regexs |
| <i>a   b   c</i> | Union (Or)      | Matches <i>a</i> or <i>b</i> or <i>c</i> , where <i>a</i> , <i>b</i> , and <i>c</i> are any regexs         |
| <i>a*</i>        | Kleene star     | Matches 0 or more <i>a</i> 's, where <i>a</i> is any regex                                                 |
| <i>a+</i>        | At least one    | Matches 1 or more <i>a</i> 's, where <i>a</i> is any regex                                                 |
| <i>a?</i>        |                 | Matches 0 or 1 <i>a</i> 's, where <i>a</i> is any regex                                                    |
| <i>a{n, m}</i>   | Replication     | Matches at least <i>n</i> but no more than <i>m</i> <i>a</i> 's, where <i>a</i> is any regex               |
| <i>a*?</i>       | Parsimonious    | Turns off greedy matching so the shortest match is selected                                                |
| <i>a+?</i>       | "               | "                                                                                                          |
| .                | Wild card       | Matches any character except newline                                                                       |
| ^                | Left anchor     | Anchors the match to the beginning of a line or string                                                     |
| \$               | Right anchor    | Anchors the match to the end of a line or string                                                           |
| [a-z]            |                 | Assuming a collating sequence, matches any single character in range                                       |
| [^a-z]           |                 | Assuming a collating sequence, matches any single character not in range                                   |
| \d               | Digit           | Matches any single digit, i.e., string in [0-9]                                                            |
| \D               | Nondigit        | Matches any single nondigit character, i.e., [^0-9]                                                        |
| \w               | Alphanumeric    | Matches any single "word" character, i.e., [a-zA-Z0-9_]                                                    |
| \W               | Nonalphanumeric | Matches any character in [^a-zA-Z0-9_]                                                                     |
| \s               | White space     | Matches any character in [space, tab, newline, etc.]                                                       |

# Regular Expressions in Perl

| <i>Syntax</i>      | <i>Name</i>      | <i>Description</i>                                                                                 |
|--------------------|------------------|----------------------------------------------------------------------------------------------------|
| <code>\s</code>    | Nonwhite space   | Matches any character not matched by <code>\s</code>                                               |
| <code>\n</code>    | Newline          | Matches newline                                                                                    |
| <code>\r</code>    | Return           | Matches return                                                                                     |
| <code>\t</code>    | Tab              | Matches tab                                                                                        |
| <code>\f</code>    | Formfeed         | Matches formfeed                                                                                   |
| <code>\b</code>    | Backspace        | Matches backspace inside []                                                                        |
| <code>\b</code>    | Word boundary    | Matches a word boundary outside []                                                                 |
| <code>\B</code>    | Nonword boundary | Matches a non-word boundary                                                                        |
| <code>\0</code>    | Null             | Matches a null character                                                                           |
| <code>\nnn</code>  | Octal            | Matches an ASCII character with octal value <i>nnn</i>                                             |
| <code>\xnn</code>  | Hexadecimal      | Matches an ASCII character with hexadecimal value <i>nn</i>                                        |
| <code>\cX</code>   | Control          | Matches an ASCII control character                                                                 |
| <code>\char</code> | Quote            | Matches <i>char</i> ; used to quote symbols such as <code>.</code> and <code>\</code>              |
| <code>(a)</code>   | Store            | Matches <i>a</i> , where <i>a</i> is any regex, and stores the matched string in the next variable |
| <code>\1</code>    | Variable         | Matches whatever the first parenthesized expression matched                                        |
| <code>\2</code>    |                  | Matches whatever the second parenthesized expression matched                                       |
| <code>...</code>   |                  | For all remaining variables                                                                        |



# Using Regular Expressions in the Real World

## Matching numbers:

`-? ([0-9]+(\.[0-9]*)? | \.[0-9]+)`

## Matching ip addresses:

`([0-9]{1,3} (\.[0-9]{1,3})){3}`

## Finding doubled words:

`([A-Za-z]+) \s+ \1`

From Friedl, J., Mastering Regular Expressions, O' Reilly, 1997.

# Simplifying Regular Expressions

Regex' s describe sets:

- Union is commutative:  $\alpha \cup \beta = \beta \cup \alpha$ .
- Union is associative:  $(\alpha \cup \beta) \cup \gamma = \alpha \cup (\beta \cup \gamma)$ .
- $\emptyset$  is the identity for union:  $\alpha \cup \emptyset = \emptyset \cup \alpha = \alpha$ .
- Union is idempotent:  $\alpha \cup \alpha = \alpha$ .

Concatenation:

- Concatenation is associative:  $(\alpha\beta)\gamma = \alpha(\beta\gamma)$ .
- $\varepsilon$  is the identity for concatenation:  $\alpha \varepsilon = \varepsilon \alpha = \alpha$ .
- $\emptyset$  is a zero for concatenation:  $\alpha \emptyset = \emptyset \alpha = \emptyset$ .

Concatenation distributes over union:

- $(\alpha \cup \beta) \gamma = (\alpha \gamma) \cup (\beta \gamma)$ .
- $\gamma (\alpha \cup \beta) = (\gamma \alpha) \cup (\gamma \beta)$ .

Kleene star:

- $\emptyset^* = \varepsilon$ .
- $\varepsilon^* = \varepsilon$ .
- $(\alpha^*)^* = \alpha^*$ .
- $\alpha^* \alpha^* = \alpha^*$ .
- $(\alpha \cup \beta)^* = (\alpha^* \beta^*)^*$ .

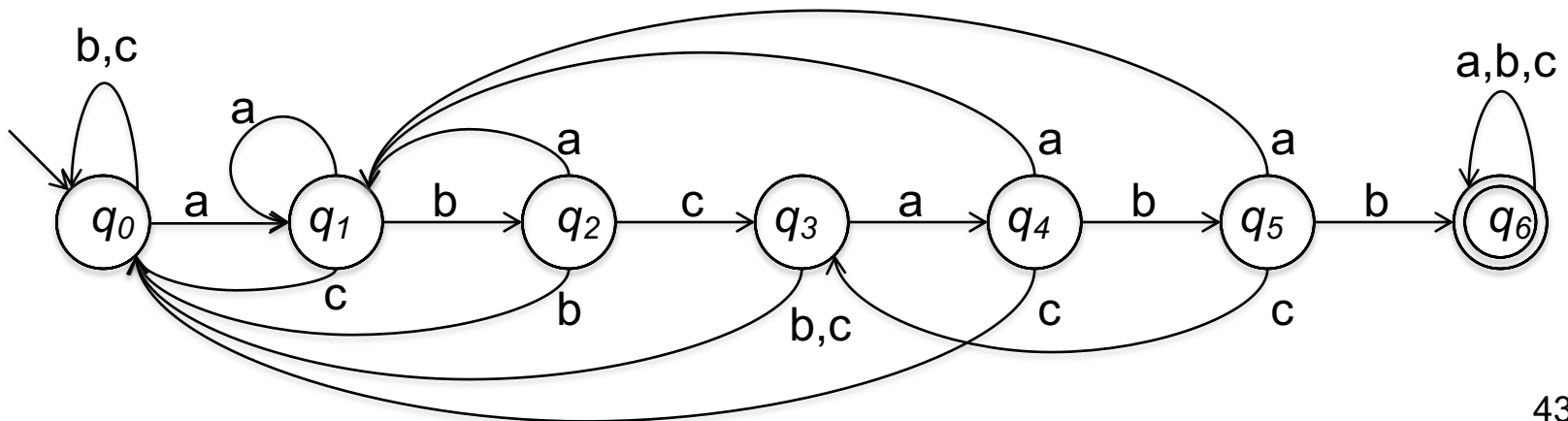
# Pattern Matching

Given a pattern  $p$  and a text  $T$ , does  $p$  occur as a substring of  $T$ ?

*Solution:* Construct a DFSA  $M$  for  $L(\Sigma^* p \Sigma^*)$ ;  $p$  occurs in  $T$  iff  $M$  accepts  $T$ .

*Example:*  $p = \text{abcabb}$

Below is the minimal DFSA that accepts  $L(\Sigma^* \text{abcabb} \Sigma^*)$



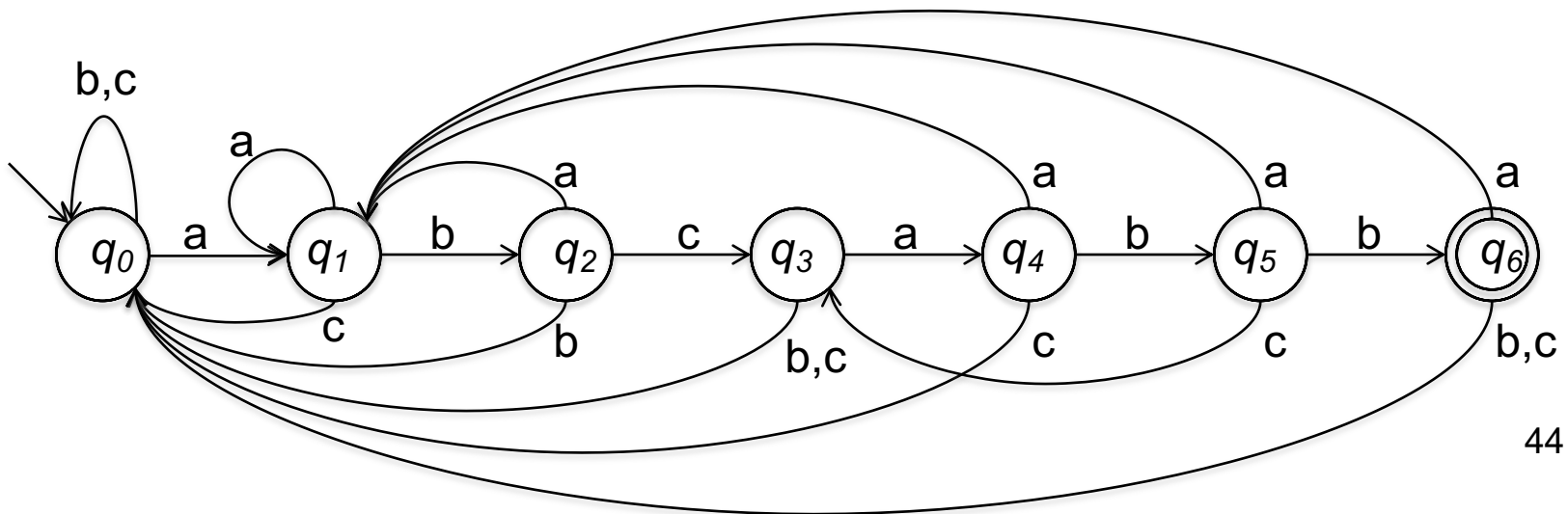
# Pattern Searching

Given a pattern  $p$  and a text  $T$ , find all occurrences of  $p$  as a substring of  $T$ .

*Solution:* Construct a DFSA  $M$  for  $L(\Sigma^* p)$  and run  $M$  on  $T$ ; an accepting state marks the end of an occurrence of  $p$ .

*Example:*  $p = \text{abcabb}$

Below is the minimal DFSA that accepts  $L(\Sigma^* \text{abcabb})$





# Multiple Patterns

## Multi-Pattern matching

Given several patterns  $p_1, p_2, \dots, p_k$ , and a text  $T$ , does any of the patterns occur as a substring of  $T$ ?

*Solution:* DFSA for  $L(\Sigma^*(p_1 \cup p_2 \cup \dots \cup p_k) \Sigma^*)$ .

## Multi-Pattern searching

Given several patterns  $p_1, p_2, \dots, p_k$ , and a text  $T$ , find all occurrences of all patterns as substrings of  $T$ .

*Solution:* DFSA for  $L(\Sigma^*(p_1 \cup p_2 \cup \dots \cup p_k))$ .